Week 7

SQL Query Implementation for E-Commerce Database

Scenario:

An e-commerce company maintains a database containing customer details, product information, orders, payments, and inventory records. The management team wants to retrieve and analyze useful information from the database to manage daily business operations effectively.

This task focuses on implementing SQL queries to retrieve, filter, sort, and search data from different database tables. It also helps generate basic business reports related to product availability, customer registrations, order status, and product performance.

By using SQL commands such as SELECT, WHERE, ORDER BY, DISTINCT, and JOIN, students can understand how to manage and analyze e-commerce data efficiently.

Required Tables:

- Customer Table
- Product Table
- Category Table
- Orders Table
- Payment Table
- Inventory Table
- Order_Items Table
- Product_Review Table

1. **Perform Basic SQL Queries:**
   1. **Display all customer details:**

      This query is used to retrieve and display all the details of customers stored in the Customer table. The

```
SQL> SELECT * FROM Customer;

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
CITY
------------------------------
          1 Arun
arun@gmail.com
Chennai

          2 Priya
priya@gmail.com
Tambaram

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
CITY
------------------------------

          3 Kaviya
kaviya@gmail.com
Chromepet

          4 Rahul
rahul@gmail.com
```

```
CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
CITY
------------------------------
Velachery

          5 Divya
divya@gmail.com
Pallavaram

          6 Vijay

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
CITY
------------------------------
vijay@gmail.com
Adyar

          7 Sneha
sneha@gmail.com
Porur


CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
CITY
------------------------------
          8 Karthik
karthik@gmail.com
```

```
         9 Meena
meena@gmail.com
Anna Nagar

CUSTOMER_ID CUSTOMER_NAME
----------- ----------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
CITY
------------------------------


        10 Ajay
ajay@gmail.com
T Nagar


10 rows selected.
```

2. Display all available products:

This query is used to retrieve and display all product details stored in the Product table. The SELECT * command displays all columns and records, including Product ID, Product Name, Price, and Category ID.

```
SQL> SELECT PRODUCT_NAME, PRICE FROM Product;

PRODUCT_NAME                                         PRICE
-------------------------------------------------- ----------
Laptop                                              55000
Mobile                                              15000
Headphone                                            2000
Keyboard                                             1500
Mouse                                                 800
Tablet                                              25000
Monitor                                             12000
Smartwatch                                           5000
Printer                                             10000
Speaker                                              3000

10 rows selected.
```

```
SQL> SELECT * FROM Product;

PRODUCT_ID PRODUCT_NAME                                        PRICE
---------- -------------------------------------------------- ----------
CATEGORY_ID
-----------
       201 Laptop                                             55000
         1

       202 Mobile                                             15000
         2

       203 Headphone                                           2000
         3


PRODUCT_ID PRODUCT_NAME                                        PRICE
---------- -------------------------------------------------- ----------
CATEGORY_ID
-----------
       204 Keyboard                                            1500
         4

       205 Mouse                                                800
         4

       206 Tablet                                             25000
         2
```

```
PRODUCT_ID PRODUCT_NAME                                        PRICE
---------- -------------------------------------------------- ----------
CATEGORY_ID
-----------
       207 Monitor                                            12000
         4

       208 Smartwatch                                          5000
         3

       209 Printer                                            10000
         5


PRODUCT_ID PRODUCT_NAME                                        PRICE
---------- -------------------------------------------------- ----------
CATEGORY_ID
-----------
       210 Speaker                                             3000
         3
```

3. Retrieve Product Names and Prices Only:
   This query is used to retrieve only the product names and their prices from the Product table. The SELECT statement specifies the required columns, PRODUCT_NAME and PRICE, instead of displaying all columns.

4. Display All Orders Placed by Customers:

This query is used to retrieve and display all orders placed by customers from the Orders table. The SELECT * command displays all columns and records, including ORDER_ID, CUSTOMER_ID, ORDER_DATE, TOTAL_AMOUNT and ORDER_STATUS.

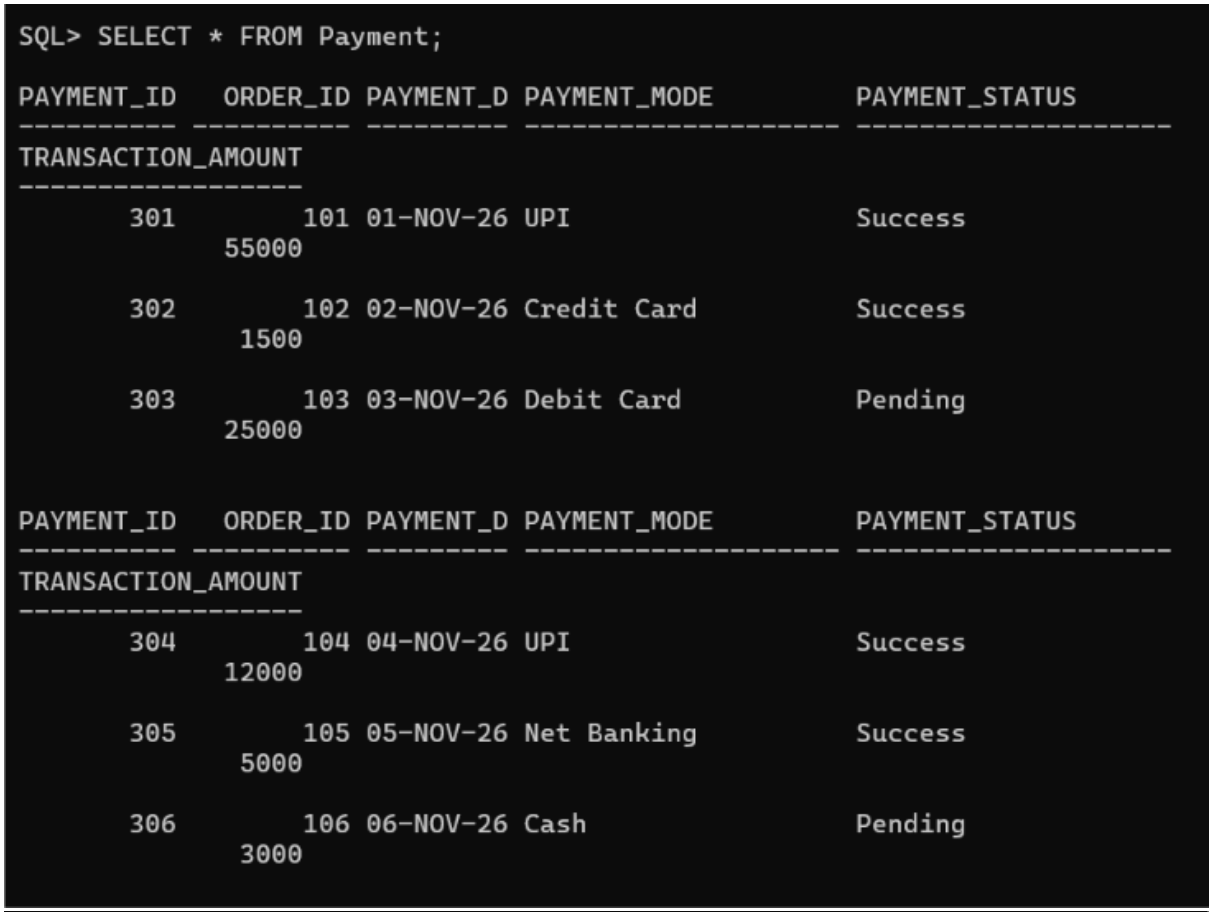
```
SQL> SELECT * FROM Orders;

  ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ----------- --------- ------------ --------------------
       101           1 01-OCT-26        55000 Delivered
       102           2 02-OCT-26        30000 Delivered
       103           3 03-OCT-26        15000 Pending
       104           4 04-OCT-26        25000 Shipped
       105           5 05-OCT-26        40000 Delivered
       106           6 06-OCT-26        12000 Pending
       107           7 07-OCT-26        18000 Shipped
       108           8 08-OCT-26        22000 Delivered
       109           9 09-OCT-26        35000 Pending
       110          10 10-OCT-26        28000 Delivered

10 rows selected.
```

5. Retrieve Payment Details:

This query is used to retrieve and display all payment details stored in the Payment table. The SELECT * command displays all columns and records, including PAYMENT_ID, ORDER_ID, PAYMENT_METHOD, AMOUNT, PAYMENT_STATUS and PAYMENT_DATE.



```
SQL> SELECT * FROM Payment;

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       301        101 01-NOV-26 UPI                  Success
             55000

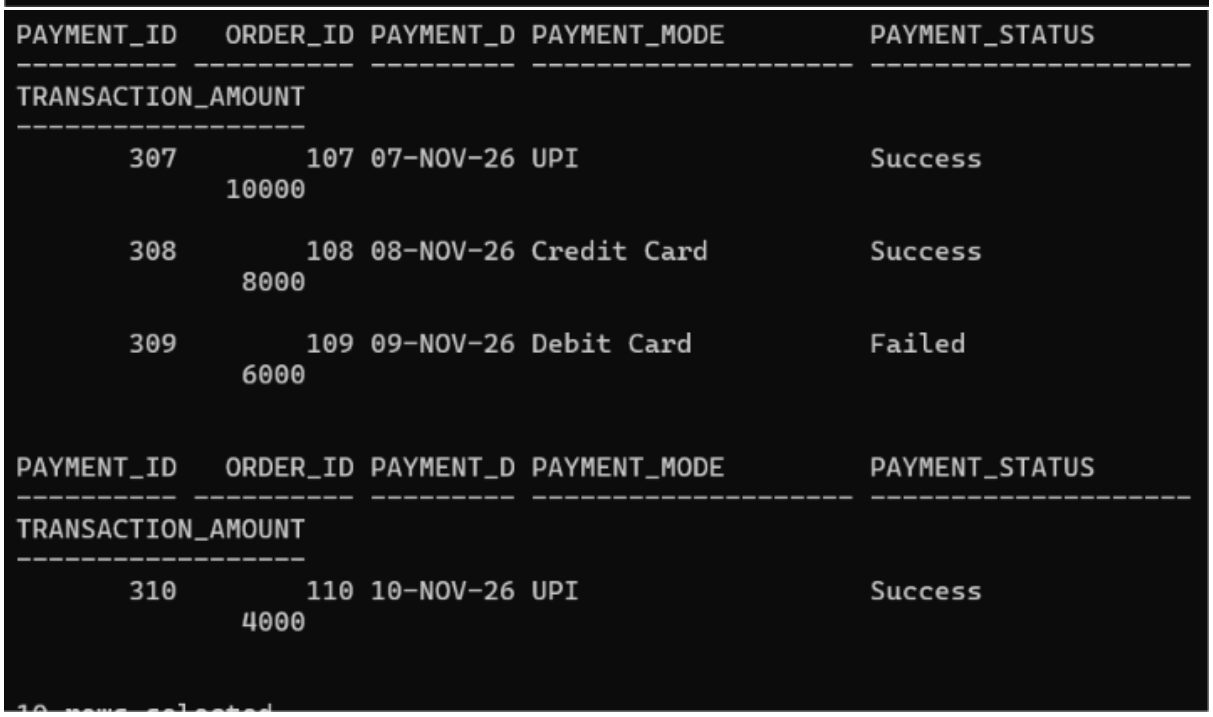
       302        102 02-NOV-26 Credit Card          Success
              1500

       303        103 03-NOV-26 Debit Card           Pending
             25000


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       304        104 04-NOV-26 UPI                  Success
             12000

       305        105 05-NOV-26 Net Banking          Success
              5000

       306        106 06-NOV-26 Cash                 Pending
              3000
```



```
PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       307        107 07-NOV-26 UPI                  Success
             10000

       308        108 08-NOV-26 Credit Card          Success
              8000

       309        109 09-NOV-26 Debit Card           Failed
              6000


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       310        110 10-NOV-26 UPI                  Success
              4000


10 rows selected.
```

2. Apply Filtering Conditions Using WHERE:

1. Find products with price greater than ₹5000:

This query is used to retrieve products whose price is greater than ₹5000 from the Product table. The WHERE clause is used to filter records based on a specific condition. Here, only products with a price above ₹5000 are displayed.

```
SQL> SELECT * FROM Product WHERE PRICE > 5000;

PRODUCT_ID PRODUCT_NAME                                        PRICE
---------- -------------------------------------------------- ----------
CATEGORY_ID
-----------
       201 Laptop                                             55000
         1

       202 Mobile                                             15000
         2

       206 Tablet                                             25000
         2


PRODUCT_ID PRODUCT_NAME                                        PRICE
---------- -------------------------------------------------- ----------
CATEGORY_ID
-----------
       207 Monitor                                            12000
         4

       209 Printer                                            10000
         5
```

2. Display Products Available in Stock:

This query is used to retrieve products that are currently available in stock. The JOIN clause connects the Product and Inventory tables using PRODUCT_ID. The WHERE clause filters products whose stock quantity is greater than zero.

```
SQL> SELECT P.* FROM Product P JOIN Inventory I ON P.PRODUCT_ID = I.PRODUCT_
ID WHERE I.STOCK_QUANTITY > 0;

PRODUCT_ID PRODUCT_NAME                                          PRICE
---------- ---------------------------------------------- ----------
CATEGORY_ID
-----------
       201 Laptop                                              55000
         1

       202 Mobile                                              15000
         2

       204 Keyboard                                             1500
         4


PRODUCT_ID PRODUCT_NAME                                          PRICE
---------- ---------------------------------------------- ----------
CATEGORY_ID
-----------
       205 Mouse                                                 800
         4
```

3. **Display Completed Orders**
   This query is used to retrieve all completed orders from the Orders table. The WHERE clause filters the records based on the ORDER_STATUS column. Only orders with the status 'Completed' are displayed.

```
SQL> SELECT * FROM Orders WHERE ORDER_STATUS = 'Delivered';

  ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ----------- --------- ------------ --------------------
       101           1 01-OCT-26        55000 Delivered
       102           2 02-OCT-26        30000 Delivered
       105           5 05-OCT-26        40000 Delivered
       108           8 08-OCT-26        22000 Delivered
       110          10 10-OCT-26        28000 Delivered
```

4. **Find Products with Rating Above 4**
   This query is used to retrieve products with a rating greater than 4 from the Product table. The WHERE clause filters the records based on the RATING column and displays only products with ratings above 4.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, R.RATING FROM Product P JOIN Produ
ct_Review R ON P.PRODUCT_ID = R.PRODUCT_ID WHERE R.RATING > 4;

PRODUCT_ID PRODUCT_NAME                                          RATING
---------- ---------------------------------------------- ----------
       201 Laptop                                                  5
       204 Keyboard                                                5
```

3. **Sort Data Using ORDER BY:**

   Tasks:

   1. Display Products from Lowest to Highest Price:

      This query is used to display all products in ascending order based on their prices. The ORDER BY clause is used to sort the records, and ASC arranges the products from the lowest price to the highest price.

```
SQL> SELECT * FROM Product ORDER BY PRICE ASC;

PRODUCT_ID PRODUCT_NAME                                        PRICE
---------- -------------------------------------------------- ----------
CATEGORY_ID
-----------
       205 Mouse                                               800
         4

       204 Keyboard                                           1500
         4

       203 Headphone                                          2000
         3


PRODUCT_ID PRODUCT_NAME                                        PRICE
---------- -------------------------------------------------- ----------
CATEGORY_ID
-----------
       210 Speaker                                            3000
         3

       208 Smartwatch                                         5000
         3

       209 Printer                                           10000
         5
```

```
PRODUCT_ID PRODUCT_NAME                                        PRICE
---------- -------------------------------------------------- ----------
CATEGORY_ID
-----------
       207 Monitor                                           12000
         4

       202 Mobile                                            15000
         2

       206 Tablet                                            25000
         2


PRODUCT_ID PRODUCT_NAME                                        PRICE
---------- -------------------------------------------------- ----------
CATEGORY_ID
-----------
       201 Laptop                                            55000
         1


10 rows selected.
```

2. **Display customers alphabetically:**
   This query is used to display all customer details in alphabetical order based on their names. The ORDER BY clause sorts the records, and ASC arranges customer names from A to Z.

```
SQL> SELECT * FROM Customer ORDER BY CUSTOMER_NAME ASC;

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
CITY
------------------------------
         10 Ajay
ajay@gmail.com
T Nagar

          1 Arun
arun@gmail.com
Chennai

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
CITY
------------------------------


          5 Divya
divya@gmail.com
Pallavaram

          8 Karthik
karthik@gmail.com
```

```
CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
CITY
------------------------------
Guindy

          3 Kaviya
kaviya@gmail.com
Chromepet

          9 Meena

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
CITY
------------------------------
meena@gmail.com
Anna Nagar

          2 Priya
priya@gmail.com
Tambaram
```

```
CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
CITY
------------------------------


          6 Vijay
vijay@gmail.com
Adyar


10 rows selected.
```

3. **Find top expensive products.**
   This query is used to display products based on their prices in descending order. The ORDER BY clause sorts the product records, and DESC arranges them from the highest price to the lowest price. This helps identify the most expensive products in the database.

```
SQL> SELECT * FROM Product ORDER BY PRICE DESC;

PRODUCT_ID PRODUCT_NAME                                        PRICE
---------- -------------------------------------------------- ----------
CATEGORY_ID
-----------
       201 Laptop                                             55000
         1

       206 Tablet                                             25000
         2

       202 Mobile                                             15000
         2


PRODUCT_ID PRODUCT_NAME                                        PRICE
---------- -------------------------------------------------- ----------
CATEGORY_ID
-----------
       207 Monitor                                            12000
         4

       209 Printer                                            10000
         5

       208 Smartwatch                                          5000
         3
```

```
PRODUCT_ID PRODUCT_NAME                                              PRICE
---------- ---------------------------------------------- ----------
CATEGORY_ID
-----------
       210 Speaker                                              3000
           3

       203 Headphone                                            2000
           3

       204 Keyboard                                             1500
           4


PRODUCT_ID PRODUCT_NAME                                              PRICE
---------- ---------------------------------------------- ----------
CATEGORY_ID
-----------
       205 Mouse                                                 800
           4


10 rows selected.
```

4. **Display latest orders first:**

   This query is used to display all customer orders based on their order dates. The ORDER BY clause sorts the records, and DESC arranges the orders from the latest date to the oldest date.

```
SQL> SELECT * FROM Orders ORDER BY ORDER_DATE DESC;

  ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ----------- --------- ------------ --------------------
       110          10 10-OCT-26        28000 Delivered
       109           9 09-OCT-26        35000 Pending
       108           8 08-OCT-26        22000 Delivered
       107           7 07-OCT-26        18000 Shipped
       106           6 06-OCT-26        12000 Pending
       105           5 05-OCT-26        40000 Delivered
       104           4 04-OCT-26        25000 Shipped
       103           3 03-OCT-26        15000 Pending
       102           2 02-OCT-26        30000 Delivered
       101           1 01-OCT-26        55000 Delivered

10 rows selected.
```

4. **Retrieve Unique Values Using DISTINCT**

Tasks:

1. **Display unique product categories:**

This query is used to retrieve unique product categories from the Product table. The DISTINCT keyword eliminates duplicate values and displays each category ID only once.

```
SQL> SELECT DISTINCT CATEGORY_ID FROM Product;

CATEGORY_ID
-----------
          1
          2
          4
          5
          3
```

2. **Find different payment methods used by customers:**

This query is used to retrieve different payment methods used by customers from the Payment table. The DISTINCT keyword eliminates duplicate payment methods and displays each payment method only once.

```
SQL> SELECT DISTINCT PAYMENT_MODE FROM Payment;

PAYMENT_MODE
--------------------
Credit Card
Net Banking
Cash
UPI
Debit Card
```

5. **Search Products Based on Conditions**

Tasks: Write SQL queries to:

1. **Find Products Within a Specific Price Range (₹20,000 to ₹40,000)**

This query is used to retrieve products whose prices are between ₹20,000 and ₹40,000 from the Product table. The BETWEEN operator is used to filter products within the specified price range, including both minimum and maximum values

```
SQL> SELECT * FROM Product WHERE PRICE BETWEEN 20000 AND 40000;

PRODUCT_ID PRODUCT_NAME                                            PRICE
---------- ---------------------------------------------- ----------
CATEGORY_ID
-----------
       206 Tablet                                              25000
         2
```

2. **Display products belonging to a particular category:**

   This query is used to retrieve products belonging to a particular category from the Product table. The WHERE clause filters the records based on the CATEGORY_ID. Here, category ID 1 is selected to display only the products belonging to that category.

```
SQL> SELECT * FROM Product WHERE CATEGORY_ID = 1;

PRODUCT_ID PRODUCT_NAME                                            PRICE
---------- ---------------------------------------------- ----------
CATEGORY_ID
-----------
       201 Laptop                                              55000
         1
```

3. **Find products currently available in inventory:**

   This query is used to retrieve products that are currently available in inventory. The JOIN clause connects the Product and Inventory tables using PRODUCT_ID. The WHERE clause filters the records based on STOCK_STATUS and displays only products with the status 'Available'.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, P.PRICE, I.STOCK_QUANTITY, I.STOCK
_STATUS FROM Product P JOIN Inventory I ON P.PRODUCT_ID = I.PRODUCT_ID WHERE
 I.STOCK_STATUS = 'Available';

PRODUCT_ID PRODUCT_NAME                                            PRICE
---------- ---------------------------------------------- ----------
STOCK_QUANT STOCK_STATU
----------- -----------
       201 Laptop                                              55000
10          Available

       202 Mobile                                              15000
5           Available

       204 Keyboard                                             1500
8           Available


PRODUCT_ID PRODUCT_NAME                                            PRICE
---------- ---------------------------------------------- ----------
STOCK_QUANT STOCK_STATU
----------- -----------
       205 Mouse                                                800
15          Available
```

4. **Search products using product name.**

   This query is used to search for products based on their names in the Product table. The LIKE operator is used to search for a specific word or pattern in the PRODUCT_NAME column. The % symbol allows matching any characters before or after the word "Mobile".

```
SQL> SELECT * FROM Product WHERE PRODUCT_NAME LIKE '%Mobile%';

PRODUCT_ID PRODUCT_NAME                                            PRICE
---------- ---------------------------------------------------- ----------
CATEGORY_ID
-----------
       202 Mobile                                                15000
         2
```

5. **Find low-stock products:**

   This query is used to identify products with low stock quantities in the inventory. The JOIN clause connects the Product and Inventory tables using PRODUCT_ID. The WHERE clause filters products whose stock quantity is less than 5.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, I.STOCK_QUANTITY, I.STOCK_STATUS F
ROM Product P JOIN Inventory I ON P.PRODUCT_ID = I.PRODUCT_ID WHERE TO_NUMBE
R(I.STOCK_QUANTITY) < 5;

PRODUCT_ID PRODUCT_NAME                                       STOCK_QUANT
---------- ---------------------------------------------------- -----------
STOCK_STATU
-----------
       202 Mobile                                                3
Low Stock
```

6. **Retrieve Customer and Product Information**
   Tasks:
   This query is used to retrieve customer details along with their order information. The JOIN clause connects the Customer and Orders tables using CUSTOMER_ID. It displays customer information and the corresponding orders placed by them.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME,C.EMAIL, O.ORDER_ID,O.ORDER_DATE,
 O.TOTAL_AMOUNT,O.ORDER_STATUS FROM Customer C JOIN Orders O ON C.CUSTOMER_I
D = O.CUSTOMER_ID;

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- --------- ------------ --------------------
          1 Arun
arun@gmail.com
       101 01-OCT-26        55000 Delivered

          2 Priya
priya@gmail.com
       102 02-OCT-26        30000 Delivered

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- --------- ------------ --------------------

          3 Kaviya
kaviya@gmail.com
       103 03-OCT-26        15000 Pending

          4 Rahul
rahul@gmail.com
```

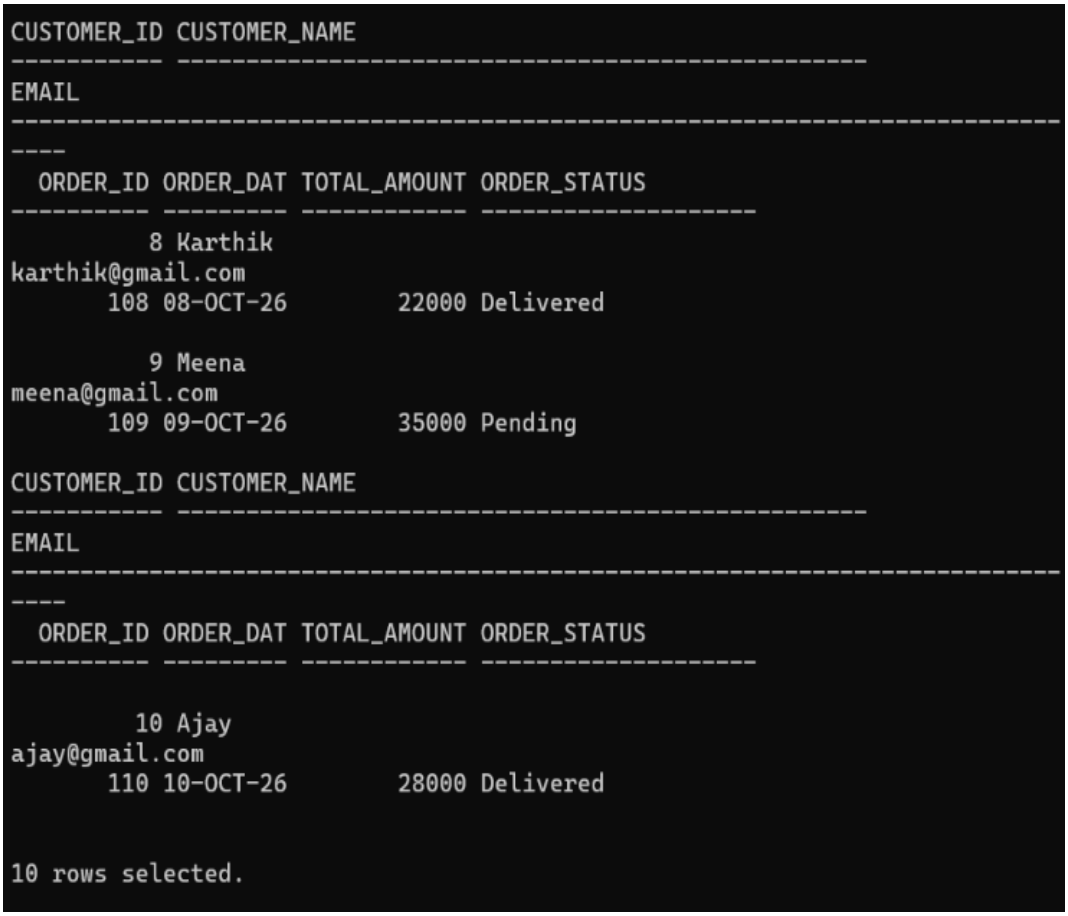
```
CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- --------- ------------ --------------------
       104 04-OCT-26        25000 Shipped

          5 Divya
divya@gmail.com
       105 05-OCT-26        40000 Delivered

          6 Vijay

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- --------- ------------ --------------------
vijay@gmail.com
       106 06-OCT-26        12000 Pending

          7 Sneha
sneha@gmail.com
       107 07-OCT-26        18000 Shipped
```



3. **Display product details with category information.**

   This query is used to retrieve product details along with their category information. The JOIN clause connects the Product and Category tables using CATEGORY_ID. It displays each product with its corresponding category name.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME,P.PRICE, C.CATEGORY_NAME FROM Produ
ct P JOIN Category C ON P.CATEGORY_ID = C.CATEGORY_ID;

PRODUCT_ID PRODUCT_NAME                                             PRICE
---------- ---------------------------------------------- ----------
CATEGORY_NAME
--------------------------------------------------
       201 Laptop                                              55000
Electronics

       202 Mobile                                              15000
Clothing

       203 Headphone                                            2000
Books


PRODUCT_ID PRODUCT_NAME                                             PRICE
---------- ---------------------------------------------- ----------
CATEGORY_NAME
--------------------------------------------------
       204 Keyboard                                             1500
Furniture

       205 Mouse                                                 800
Furniture

       206 Tablet                                              25000
Clothing
```

```
PRODUCT_ID PRODUCT_NAME                                             PRICE
---------- ---------------------------------------------- ----------
CATEGORY_NAME
--------------------------------------------------
       207 Monitor                                             12000
Furniture

       208 Smartwatch                                           5000
Books

       209 Printer                                             10000
Accessories


PRODUCT_ID PRODUCT_NAME                                             PRICE
---------- ---------------------------------------------- ----------
CATEGORY_NAME
--------------------------------------------------
       210 Speaker                                              3000
Books


10 rows selected.
```

4. **Find customers who purchased a specific product**

   This query is used to identify customers who purchased a specific product. The JOIN clause connects the Customer, Orders, Order_Items and Product tables using their corresponding ID columns. The WHERE clause filters the records based on the selected PRODUCT_ID (201). The DISTINCT keyword avoids duplicate customer records.

```
SQL> SELECT DISTINCT C.CUSTOMER_ID, C.CUSTOMER_NAME,P.PRODUCT_NAME FROM Cust
omer C JOIN Orders O ON C.CUSTOMER_ID = O.CUSTOMER_ID JOIN Order_Items OI ON
 O.ORDER_ID = OI.ORDER_ID JOIN Product P ON OI.PRODUCT_ID = P.PRODUCT_ID WHE
RE P.PRODUCT_ID = 201;

CUSTOMER_ID CUSTOMER_NAME
----------- ----------------------------------------------
PRODUCT_NAME
----------------------------------------------
          1 Arun
Laptop
```

5. **Display products purchased by each customer.**

   This query is used to display the products purchased by each customer. The JOIN clause connects the Customer, Orders, Order_Items and Product tables using their corresponding ID columns. It retrieves customer details along with the products they purchased and their quantities.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME,O.ORDER_ID, P.PRODUCT_ID, P.PRODU
CT_NAME, OI.QUANTITY FROM Customer C JOIN Orders O ON C.CUSTOMER_ID = O.CUST
OMER_ID JOIN Order_Items OI ON O.ORDER_ID = OI.ORDER_ID JOIN Product P ON OI
.PRODUCT_ID = P.PRODUCT_ID;

CUSTOMER_ID CUSTOMER_NAME                                    ORDER_ID
----------- ---------------------------------------------- ----------
PRODUCT_ID PRODUCT_NAME                                     QUANTITY
---------- ---------------------------------------------- ----------
         1 Arun                                                  101
       201 Laptop                                                  1

         2 Priya                                                 102
       202 Mobile                                                  2

         3 Kaviya                                                103
       203 Headphone                                               3


CUSTOMER_ID CUSTOMER_NAME                                    ORDER_ID
----------- ---------------------------------------------- ----------
PRODUCT_ID PRODUCT_NAME                                     QUANTITY
---------- ---------------------------------------------- ----------
         4 Rahul                                                 104
       204 Keyboard                                                2

         5 Divya                                                 105
       205 Mouse                                                   4

         6 Vijay                                                 106
       206 Tablet                                                  1

CUSTOMER_ID CUSTOMER_NAME                                    ORDER_ID
----------- ---------------------------------------------- ----------
PRODUCT_ID PRODUCT_NAME                                     QUANTITY
---------- ---------------------------------------------- ----------
         7 Sneha                                                 107
       207 Monitor                                                 2

         8 Karthik                                               108
       208 Smartwatch                                              3

         9 Meena                                                 109
       209 Printer                                                 1


CUSTOMER_ID CUSTOMER_NAME                                    ORDER_ID
----------- ---------------------------------------------- ----------
PRODUCT_ID PRODUCT_NAME                                     QUANTITY
---------- ---------------------------------------------- ----------
        10 Ajay                                                  110
       210 Speaker                                                 2


10 rows selected.
```

7.Apply Multiple Filtering Conditions

Tasks:

1. **Find electronics products costing more than ₹10000.**

- JOIN combines the Product and Category tables.
- CATEGORY_NAME = 'Electronics' filters electronics products.
- PRICE > 10000 displays products costing more than ₹10,000.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, P.PRICE, C.CATEGORY_NAME FROM Prod
uct P JOIN Category C ON P.CATEGORY_ID = C.CATEGORY_ID WHERE C.CATEGORY_NAME
 = 'Electronics' AND P.PRICE > 10000;

PRODUCT_ID PRODUCT_NAME                                           PRICE
---------- ---------------------------------------------- ----------
CATEGORY_NAME
--------------------------------------------------
       201 Laptop                                             55000
Electronics
```

2. **Search products containing the word "Mobile".**

- LIKE is used to search for a specific word or pattern.
- % represents any number of characters.
- '%Mobile%' searches for product names containing the word "Mobile" anywhere in the name.

```
SQL> SELECT * FROM Product WHERE PRODUCT_NAME LIKE '%Mobile%';

PRODUCT_ID PRODUCT_NAME                                           PRICE
---------- ---------------------------------------------- ----------
CATEGORY_ID
-----------
       202 Mobile                                             15000
         2
```

3. **Find orders placed within a specific date range.**

- BETWEEN is used to filter orders within a specific date range.
- TO_DATE() converts the given date into Oracle date format.
- This query retrieves orders placed between 1 January 2026 and 31 December 2026.

```
SQL> SELECT * FROM Orders WHERE ORDER_DATE BETWEEN TO_DATE('01-01-2026', 'DD
-MM-YYYY') AND TO_DATE('31-12-2026', 'DD-MM-YYYY');

  ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ----------- --------- ------------ --------------------
       101           1 01-OCT-26        55000 Delivered
       102           2 02-OCT-26        30000 Delivered
       103           3 03-OCT-26        15000 Pending
       104           4 04-OCT-26        25000 Shipped
       105           5 05-OCT-26        40000 Delivered
       106           6 06-OCT-26        12000 Pending
       107           7 07-OCT-26        18000 Shipped
       108           8 08-OCT-26        22000 Delivered
       109           9 09-OCT-26        35000 Pending
       110          10 10-OCT-26        28000 Delivered

10 rows selected.
```

8.Generate Basic Business Reports

Report 1: Product Availability Report

- JOIN combines the Product and Inventory tables.
- PRODUCT_NAME displays the product name.
- PRICE displays the product price.
- STOCK_QUANTITY displays the available stock quantity.
- STOCK_STATUS displays the availability status.

```
SQL> SELECT P.PRODUCT_NAME,P.PRICE,I.STOCK_QUANTITY,I.STOCK_STATUS FROM Prod
uct P JOIN Inventory I ON P.PRODUCT_ID = I.PRODUCT_ID;

PRODUCT_NAME                                         PRICE STOCK_QUANT
-------------------------------------------------- ---------- ----------
STOCK_STATU
-----------
Laptop                                                55000 10
Available

Mobile                                                15000 3
Low Stock

Keyboard                                               1500 8
Available


PRODUCT_NAME                                         PRICE STOCK_QUANT
-------------------------------------------------- ---------- ----------
STOCK_STATU
-----------
Mouse                                                  800 15
Available
```

Report 2: Customer Report
Generate:

- COUNT(*) counts the total number of records in the Customer table.
- AS TOTAL_CUSTOMERS gives a meaningful name to the output column.

```
SQL> SELECT COUNT(*) AS TOTAL_CUSTOMERS FROM Customer;

TOTAL_CUSTOMERS
---------------
             10
```

New customer registrations:

- REGISTRATION_DATE displays the date on which the customer registered.
- WHERE filters customers registered between 1 January 2026 and 31 December 2026.
- ORDER BY REGISTRATION_DATE DESC sorts the latest customer registrations first.

```
SQL> SELECT CUSTOMER_ID,CUSTOMER_NAME,EMAIL,REGISTRATION_DATE AS CREATED_AT
FROM Customer WHERE REGISTRATION_DATE >= DATE '2026-01-01' AND REGISTRATION_
DATE < DATE '2027-01-01' ORDER BY REGISTRATION_DATE DESC;

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
CREATED_A
---------
          1 Arun
arun@gmail.com
03-OCT-26

          2 Priya
priya@gmail.com
08-JUL-26

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
----
CREATED_A
---------

          3 Kaviya
kaviya@gmail.com
17-APR-26
```

Report 3: Order Report
Display:
Total orders:

- COUNT(*) counts the total number of records in the Orders table.
- AS TOTAL_ORDERS gives a meaningful name to the output column.

```
SQL> SELECT COUNT(*) AS TOTAL_ORDERS FROM Orders;

TOTAL_ORDERS
------------
          10
```

Completed orders:

- COUNT(*) counts the total number of orders.
- ORDER_STATUS = 'Delivered' selects only successfully delivered orders.
- AS COMPLETED_ORDERS displays the result with a meaningful column name.

```
SQL> SELECT COUNT(*) AS COMPLETED_ORDERS FROM Orders WHERE ORDER_STATUS = 'D
elivered';

COMPLETED_ORDERS
----------------
               5
```

Pending orders:

- COUNT(*) counts the total number of orders.
- ORDER_STATUS = 'Pending' filters only pending orders.
- AS PENDING_ORDERS gives a meaningful name to the output column.

```
SQL> SELECT COUNT(*) AS PENDING_ORDERS FROM Orders WHERE ORDER_STATUS = 'Pen
ding';

PENDING_ORDERS
--------------
             3
```

Cancelled orders:

- COUNT(*) counts the total number of orders.
- ORDER_STATUS = 'Cancelled' filters cancelled orders.
- AS CANCELLED_ORDERS gives a meaningful name to the output column.

```
SQL> SELECT COUNT(*) AS CANCELLED_ORDERS FROM Orders WHERE ORDER_STATUS = 'C
ancelled';

CANCELLED_ORDERS
----------------
               0
```

Report 4: Product Performance Report
Find:

Highest-priced products:

- MAX(PRICE) finds the highest product price.
- The subquery retrieves the maximum price.
- The outer query displays the details of the product with the highest price.

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, PRICE FROM Product WHERE PRICE = (SELE
CT MAX(PRICE) FROM Product);

PRODUCT_ID PRODUCT_NAME                                           PRICE
---------- ---------------------------------------------- ----------
       201 Laptop                                              55000
```

Most reviewed products:

- JOIN combines Product and Product_Review tables.
- COUNT(R.REVIEW_ID) counts the reviews for each product.
- GROUP BY groups reviews by product.
- ORDER BY TOTAL_REVIEWS DESC displays the most reviewed products first.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, COUNT(R.REVIEW_ID) AS TOTAL_REVIEW
S FROM Product P JOIN Product_Review R ON P.PRODUCT_ID = R.PRODUCT_ID GROUP
BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY TOTAL_REVIEWS DESC;

PRODUCT_ID PRODUCT_NAME                                    TOTAL_REVIEWS
---------- ---------------------------------------------- -------------
       201 Laptop                                                     1
       204 Keyboard                                                   1
       203 Headphone                                                  1
       202 Mobile                                                     1
       205 Mouse                                                      1
```

Available products:

- JOIN combines Product and Inventory tables.
- STOCK_QUANTITY displays the available stock.
- STOCK_STATUS = 'Available' filters products that are available.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, I.STOCK_QUANTITY, I.STOCK_STATUS F
ROM Product P JOIN Inventory I ON P.PRODUCT_ID = I.PRODUCT_ID WHERE I.STOCK_
STATUS = 'Available';

PRODUCT_ID PRODUCT_NAME                                       STOCK_QUANT
---------- -------------------------------------------------- -----------
STOCK_STATU
-----------
       201 Laptop                                            10
Available

       204 Keyboard                                          8
Available

       205 Mouse                                             15
Available
```

Conclusion

Thus, the E-Commerce Database project was successfully completed using SQL queries. Various operations such as data retrieval, filtering, sorting, joining tables, and generating business reports were performed. Customer details, product information, order status, payment details, and product availability were analyzed successfully. This project helped in understanding the practical implementation of SQL concepts in database management.